

Group Project Cheat Sheet

Main → Branch → Commit & Push → Pull Request → Review → Claude Code ile Merge → Final Main

Amaç: İki kişi aynı GitHub reposunda birbirinin çalışmasını bozmadan geliştirme yapsın. Her öğrenci kendi branch'inde çalışsın; proje tamamen bittiğinde değişiklikler review edilip main'de birleştirilsin.

1 Repo	main	student1 / student2	PR + Review	Claude Merge	Final main
--------	------	---------------------	-------------	--------------	------------

1. Tek GitHub reposu oluşturun

İkinizden biri Repo Owner olsun. Repo Owner GitHub → Repositories → New ile boş repository oluşturur. GitHub'ın "...or create a new repository on the command line" komutlarını kopyalayın.

2. Local projeyi GitHub'a bağlayın

Repo Owner, VS Code'da proje klasörünü açar ve GitHub'dan kopyaladığı komutları terminalde run'lar.

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin REPO_URL
git push -u origin main
```

3. Takım arkadaşını collaborator olarak ekleyin

Repo Owner: GitHub → Repository → Settings → Collaborators → Add people. Diğer öğrenciyi ekleyin; diğer öğrenci daveti Accept etsin.

Önemli: 2 öğrenci = 1 repository. İki ayrı repo oluşturmayın.

4. Ortak başlangıç sürümünü hazırlayın

Branch'lere ayrılmadan önce ortak başlangıç setup'ını birlikte tamamlayın ve localhost'ta çalıştığını kontrol edin.

```
git add .
git commit -m "shared project setup"
git push origin main
```

Önemli: Bu, branch'lere ayrılmadan önce yapılan tek ortak main push'udur. student1 ve student2 branch'leri oluşturulduktan sonra merge bitene kadar git push origin main kullanmayın.

5. İkinci öğrenci projeyi bilgisayarına alsın

GitHub → Repository → Code → HTTPS. Repo URL'ini kopyalayın ve ikinci öğrencinin terminalinde run'layın:

```
git clone REPO_URL
```

Oluşan klasörü VS Code ile açın. Gerekirse npm install ve npm run dev ile localhost'u kontrol edin.

Branch'lere Ayrılın

6. İkkiniz de önce latest main üzerinde olun

Her öğrenci kendi terminalinde:

```
git checkout main
git pull origin main
git branch
```

Yıldız main'in yanında olmalı:

```
* main
```

7. Kendi branch'inizi oluşturun

Student 1:

```
git checkout -b student1
```

Student 2:

```
git checkout -b student2
```

Kontrol:

```
git branch
```

Student 1 örneği:

```
main
* student1
```

Önemli: Yıldız hangi branch'te çalıştığını gösterir. Feature geliştirirken yıldız main yanında olmamalıdır.

8. Branch'inizi GitHub'a gönderin

Student 1:

```
git push -u origin student1
```

Student 2:

```
git push -u origin student2
```

GitHub'daki branch listesinden kendi branch'inizi gördüğünüzü kontrol edin.

9. Proje boyunca kendi branch'inizde kalın

Student 1 bütün projeyi student1 branch'inde, Student 2 bütün projeyi student2 branch'inde geliştirir. Kod yazmaya başlamadan önce:

```
git branch
```

Önemli çalışan değişikliklerden sonra kendi branch'inize push edin:

Student 1:

```
git add .
git commit -m "yaptığın değişiklik"
git push origin student1
```

Student 2:

```
git add .
git commit -m "yaptığın değişiklik"
git push origin student2
```

Önemli: Branch'ler oluşturulduktan sonra merge bitene kadar git push origin main kullanmayın. Student 1 → student1, Student 2 → student2 branch'ine push eder.

Proje Bittiğinde

Son test → Pull Request → Review → Claude Code ile Merge → Final main

10. Kendi branch'inizi son kez test edin

Her öğrenci kendi branch'inde kendi sorumluluklarını localhost'ta test eder. Her şey hazırsa:

Önce localhost'ta test edin, sonra kendi branch'inize son commit + push yapın:

```
npm run dev
git add .
git commit -m "projeyi tamamla"
# Student 1:
git push origin student1
# Student 2:
git push origin student2
```

11. İkiniz de Pull Request açın

GitHub → Compare & pull request. base mutlaka main olmalı:

```
Student 1: base: main ← compare: student1
Student 2: base: main ← compare: student2
```

Kısa bir başlık ve yaptığınız değişiklikleri yazın → Create Pull Request.

12. Birbirinizin PR'ını review edin

GitHub → Pull Request → Files changed. Birbirinizin PR'ında şunları kontrol edin:

- Yanlış dosya silinmiş mi veya .env eklenmiş mi?
- Diğer öğrencinin yaptığı alanlar bozulmuş mu?
- Kendi sorumluluğundaki iş tamamlanmış mı?

Problem varsa comment bırakın. Her şey doğruysa Review changes → Approve.

Önemli: PR approve edilmeden merge etmeyin.

13. Merge işlemini Claude Code'a yaptırın

İki PR da approve edildikten sonra Repo Owner terminalde claude run'lar ve şunu söyler:

```
student1 ve student2 PR'ları review edildi.
Latest main'i al; önce student1, sonra student2 branch'ini main'e merge et.
Conflict varsa iki öğrencinin çalışan özelliklerini de koru ve projeyi test et.
Force push yapma. Her şey çalışıyorsa final main'i GitHub'a push et.
```

14. Conflict çıkarsa Claude Code ile çözün

CONFLICT çıkması normaldir. Rastgele Accept Current / Accept Incoming seçmeyin. Claude Code iki öğrencinin yaptığı işi de koruyarak çözün; sonra npm run dev ile final projeyi tekrar test edin.

15. İkiniz de final main'i alın

```
git checkout main
git pull origin main
```

Final birleşmiş projeyi birlikte test edin ve sadece main branch'ini deploy edin.

Altın Kurallar

- 2 öğrenci = 1 GitHub repo.
- Student 1 → student1, Student 2 → student2 branch'inde çalışır.
- Feature geliştirirken direkt main üzerinde çalışma.
- Push: Student 1 → git push origin student1; Student 2 → git push origin student2.
- Branch'ler açıldıktan sonra merge bitene kadar git push origin main kullanma.
- Proje bitince PR + review yapın; merge/conflict çözümünü Claude Code'a yaptırın.
- Merge bittikten sonra ikiniz de final main'i pull edin.